

**Laxmi Charitable Trust's**  
**Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce**  
**Department of Information Technology (B.Sc.I.T Semester V)**  
**Applied Business Analytics**

## Practical V

|                              |                                   |
|------------------------------|-----------------------------------|
| Roll No.: T012               | Name: Suyash Karnale              |
| Class: TYIT                  | Batch: 1                          |
| Date of Assignment: 21-08-26 | Date/Time of Submission: 21-08-26 |

A.Roll a dice 1,000 times and show first 20 outcomes.

Calculate experimental probability of each face (1–6).

Compare with theoretical probability ( $1/6$  each).

Find probabilities of:

- Each number (1–6)
- Even numbers
- Odd numbers
- Numbers  $> 4$
- Numbers  $< 4$
- Numbers  $\geq 4$

Visualize results with charts:

- Bar chart: frequency of 1–6
- Bar chart: experimental probability of 1–6
- Pie chart: percentage distribution of outcomes
- Line chart: experimental vs theoretical probability (1–6)
- Chart: frequency of Even vs Odd
- Chart: experimental probability of Even, Odd,  $>4$ ,  $<4$

```
import numpy as np

import matplotlib.pyplot as plt

print("===== A) DICE PROBABILITY EXPERIMENT =====")

# 1. Simulate 1000 dice rolls

rolls = np.random.randint(1, 7, 1000)

print("\nFirst 20 dice outcomes:")

print(rolls[:20])

# Frequency of each number
```

```
frequency = []

for i in range(1, 7):

    count = np.sum(rolls == i)

    frequency.append(count)

# 2. Experimental probability

print("\nExperimental Probability:")

for i in range(6):

    print(i + 1, "=", frequency[i] / 1000)

# 3. Theoretical probability

print("\nTheoretical Probability:")

for i in range(1, 7):

    print(i, "=", 1 / 6)

# 4. Probability of getting 1 to 6

print("\nProbability of getting each number:")

for i in range(6):

    print("P(", i + 1, ") =", frequency[i] / 1000)

# 5. Even number

even = np.sum(rolls % 2 == 0)

print("\n5. Probability of Even Number =", even / 1000)

# 6. Odd number

odd = np.sum(rolls % 2 != 0)

print("\n6. Probability of Odd Number =", odd / 1000)

# 7. Greater than 4

greater4 = np.sum(rolls > 4)

print("\n7. Probability of Number > 4 =", greater4 / 1000)
```

```

# 8. Less than 4

less4 = np.sum(rolls < 4)

print("8. Probability of Number < 4 =", less4 / 1000)

# 9. Greater than or equal to 4

greater_equal4 = np.sum(rolls >= 4)

print("9. Probability of Number >= 4 =", greater_equal4 / 1000)

# 10. Bar chart - Frequency

plt.bar(range(1, 7), frequency)

plt.xlabel("Dice Number")

plt.ylabel("Frequency")

plt.title("Frequency of Dice Outcomes")

plt.show()

# 11. Bar chart - Experimental Probability

probability = np.array(frequency) / 1000

plt.bar(range(1, 7), probability)

plt.xlabel("Dice Number")

plt.ylabel("Probability")

plt.title("Experimental Probability")

plt.show()

# 12. Pie chart

plt.pie(

    frequency,

    labels=["1", "2", "3", "4", "5", "6"],

    autopct="%1.1f%%"

)

```

```
plt.title("Dice Outcome Percentage")

plt.show()

# 13. Experimental vs Theoretical

theoretical = [1 / 6] * 6

plt.plot(

    range(1, 7),

    probability,

    marker="o",

    label="Experimental"

)

plt.plot(

    range(1, 7),

    theoretical,

    marker="o",

    label="Theoretical"

)

plt.xlabel("Dice Number")

plt.ylabel("Probability")

plt.title("Experimental vs Theoretical Probability")

plt.legend()

plt.grid()

plt.show()

# 14. Even vs Odd

plt.bar(

    ["Even", "Odd"],
```

```
[even, odd]

)

plt.xlabel("Outcome")

plt.ylabel("Frequency")

plt.title("Even vs Odd Outcomes")

plt.show()

# 15. Experimental probability categories

categories = [

    "Even",

    "Odd",

    "> 4",

    "< 4"

]

values = [

    even / 1000,

    odd / 1000,

    greater4 / 1000,

    less4 / 1000

]

plt.bar(categories, values)

plt.xlabel("Category")

plt.ylabel("Probability")

plt.title("Experimental Probability")

plt.show()
```

## OUTPUT:

```
First 20 dice outcomes:  
[1 6 4 1 4 5 3 6 3 5 5 1 2 6 5 3 5 5 1 4]
```

### Experimental Probability:

```
1 = 0.173  
2 = 0.16  
3 = 0.167  
4 = 0.162  
5 = 0.162  
6 = 0.176
```

### Theoretical Probability:

```
1 = 0.16666666666666666  
2 = 0.16666666666666666  
3 = 0.16666666666666666  
4 = 0.16666666666666666  
5 = 0.16666666666666666  
6 = 0.16666666666666666
```

### Probability of getting each number:

```
P( 1 ) = 0.173
```

```
P( 2 ) = 0.16
```

```
P( 3 ) = 0.167
```

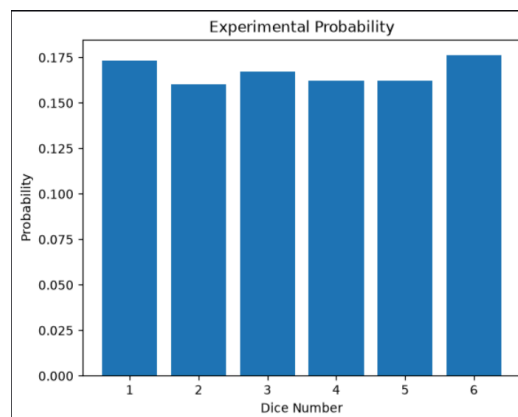
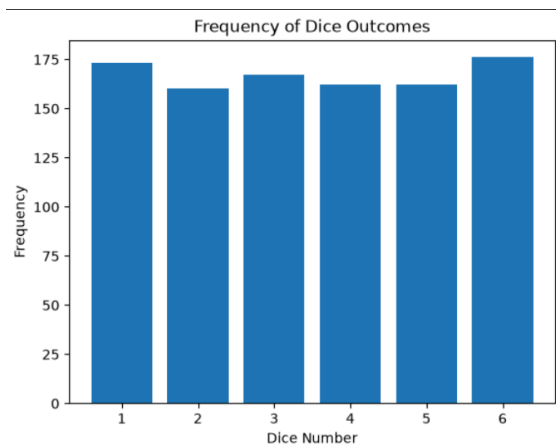
```
...
```

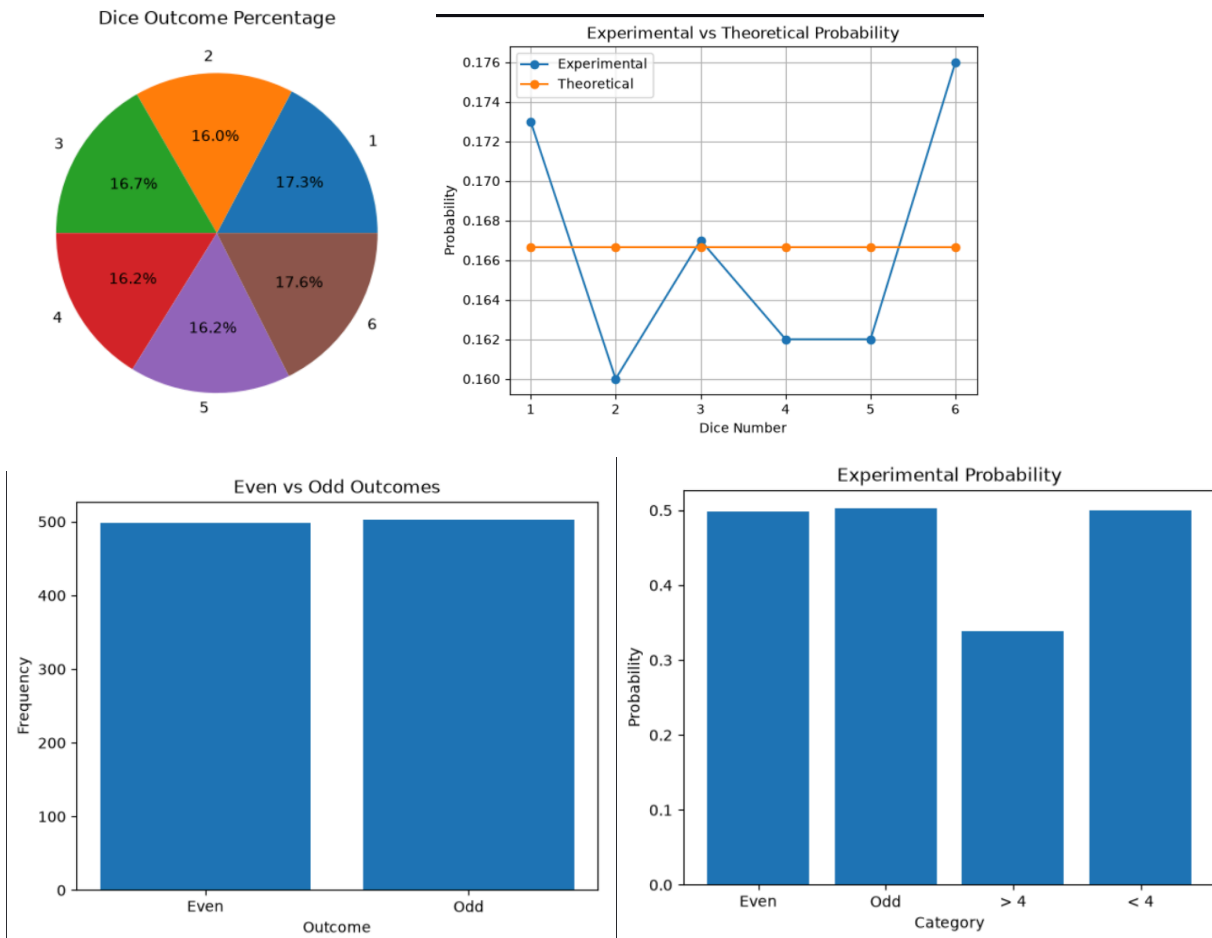
```
6. Probability of Odd Number = 0.502
```

```
7. Probability of Number > 4 = 0.338
```

```
8. Probability of Number < 4 = 0.5
```

```
9. Probability of Number >= 4 = 0.5
```





## B. Generate random numbers from Normal, Uniform, and Exponential distributions and analyze their properties:

1. Generate 1,000 random numbers from Normal distribution (mean=50, SD=10).
2. Compute descriptive statistics: mean, median, mode, variance, SD.
3. Find: mean, median, variance, SD, min, max.
4. Plot histogram and analyze shape.
5. Create box plot.
6. Calculate % values between 40–60.
7. Calculate % values between 30–70.
8. Generate 1,000 random numbers from Uniform distribution (0–100).
9. Plot histogram.
10. Create box plot.
11. Generate 1,000 random numbers from Exponential distribution (scale=2).
12. Plot histogram.
13. Create box plot and check extreme values.
14. Calculate % values > 2.
15. Calculate % values > 5.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
```

```

from scipy import stats

print("===== B) PROBABILITY DISTRIBUTIONS =====")

# =====
# NORMAL DISTRIBUTION
# =====

print("\n----- NORMAL DISTRIBUTION -----")

# 1. Generate 1000 numbers
normal_data = np.random.normal(
    50,
    10,
    1000
)

print("Mean = 50")
print("Standard Deviation = 10")

# 2. Mean, Median, Mode, Variance, SD
mean = np.mean(normal_data)
median = np.median(normal_data)
mode = stats.mode(normal_data, keepdims=True).mode[0]
variance = np.var(normal_data)
std = np.std(normal_data)

print("\n2. Statistics")
print("Mean =", mean)
print("Median =", median)
print("Mode =", mode)
print("Variance =", variance)
print("Standard Deviation =", std)

# 3. Statistics
print("\n3. Normal Distribution Statistics")
print("Mean =", mean)
print("Median =", median)
print("Variance =", variance)
print("Standard Deviation =", std)
print("Minimum =", np.min(normal_data))
print("Maximum =", np.max(normal_data))

# 4. Histogram
plt.hist(normal_data, bins=30)
plt.xlabel("Value")
plt.ylabel("Frequency")

```



```

plt.title("Normal Distribution Histogram")
plt.show()

print("\nAnalysis:")
print("The normal distribution is approximately bell-shaped.")
print("Most values are around the mean 50.")

# 5. Box plot
plt.boxplot(normal_data)
plt.ylabel("Value")
plt.title("Normal Distribution Box Plot")
plt.show()

# 6. Percentage between 40 and 60
percentage_40_60 = np.mean(
    (normal_data >= 40) &
    (normal_data <= 60)
) * 100

print("\n6. Percentage between 40 and 60 =",
      percentage_40_60, "%")

# 7. Percentage between 30 and 70
percentage_30_70 = np.mean(
    (normal_data >= 30) &
    (normal_data <= 70)
) * 100

print("7. Percentage between 30 and 70 =",
      percentage_30_70, "%")

# =====
# UNIFORM DISTRIBUTION
# =====

print("\n----- UNIFORM DISTRIBUTION -----")

# 8. Generate 1000 numbers between 0 and 100
uniform_data = np.random.uniform(
    0,
    100,
    1000
)

print("Number of values =", len(uniform_data))
print("Minimum =", np.min(uniform_data))

```

```

print("Maximum =", np.max(uniform_data))
print("Mean =", np.mean(uniform_data))

# 9. Histogram
plt.hist(uniform_data, bins=20)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Uniform Distribution Histogram")
plt.show()

# 10. Box plot
plt.boxplot(uniform_data)
plt.ylabel("Value")
plt.title("Uniform Distribution Box Plot")
plt.show()

# =====
# EXPONENTIAL DISTRIBUTION
# =====
print("\n----- EXPONENTIAL DISTRIBUTION -----")

# 11. Generate 1000 values
exponential_data = np.random.exponential(
    scale=2,
    size=1000
)
print("Scale = 2")
print("Number of values =", len(exponential_data))
print("Mean =", np.mean(exponential_data))

# 12. Histogram
plt.hist(exponential_data, bins=30)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Exponential Distribution Histogram")
plt.show()

# 13. Box plot
plt.boxplot(exponential_data)
plt.ylabel("Value")
plt.title("Exponential Distribution Box Plot")
plt.show()

# Find possible extreme values
Q1 = np.percentile(exponential_data, 25)
Q3 = np.percentile(exponential_data, 75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

extreme = exponential_data[

```

```

        (exponential_data < lower) |
        (exponential_data > upper)
    ]

print("\n13. Extreme Values")
print("Q1 =", Q1)
print("Q3 =", Q3)
print("IQR =", IQR)
print("Possible extreme values =", len(extreme))

# 14. Percentage greater than 2
greater2 = np.mean(exponential_data > 2) * 100

print("\n14. Percentage greater than 2 =",
      greater2, "%")

# 15. Percentage greater than 5
greater5 = np.mean(exponential_data > 5) * 100

print("15. Percentage greater than 5 =",
      greater5, "%")

```

OUTPUT:

```

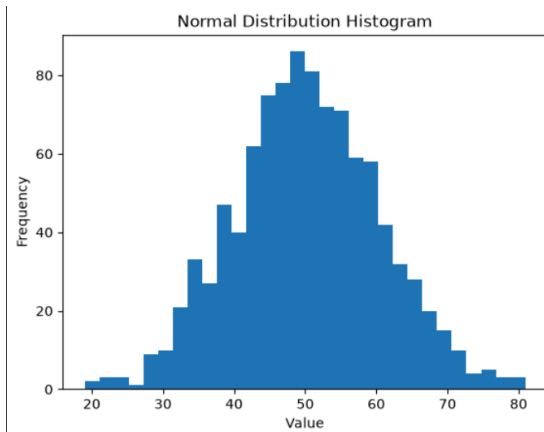
===== B) PROBABILITY DISTRIBUTIONS =====

----- NORMAL DISTRIBUTION -----
Mean = 50
Standard Deviation = 10

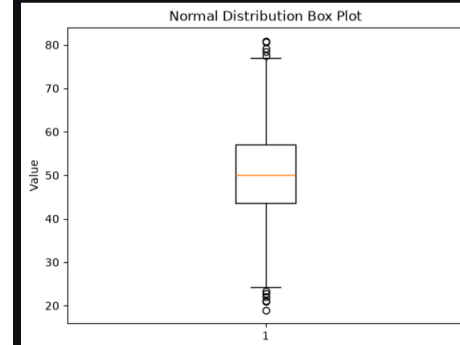
2. Statistics
Mean = 50.23357445164143
Median = 50.035564491325644
Mode = 18.99933260476886
Variance = 104.57052337524222
Standard Deviation = 10.22597297939136

3. Normal Distribution Statistics
Mean = 50.23357445164143
Median = 50.035564491325644
Variance = 104.57052337524222
Standard Deviation = 10.22597297939136
Minimum = 18.99933260476886
Maximum = 80.91911369101841

```

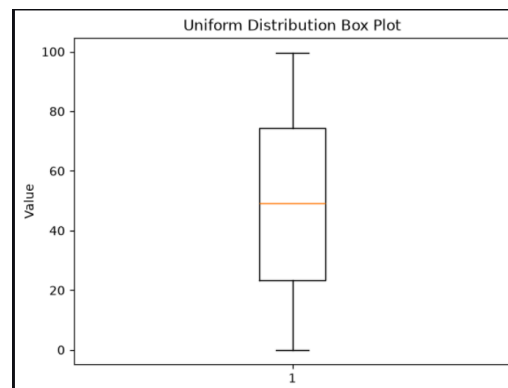
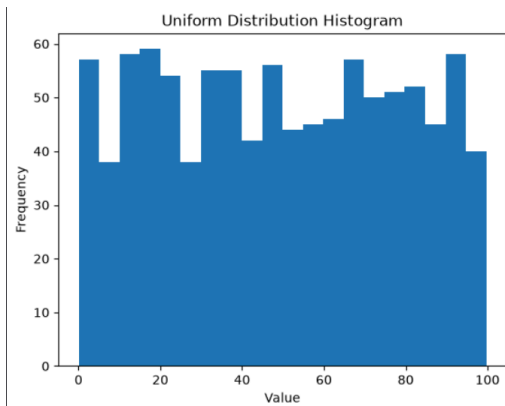


Analysis:  
The normal distribution is approximately bell-shaped.  
Most values are around the mean 50.

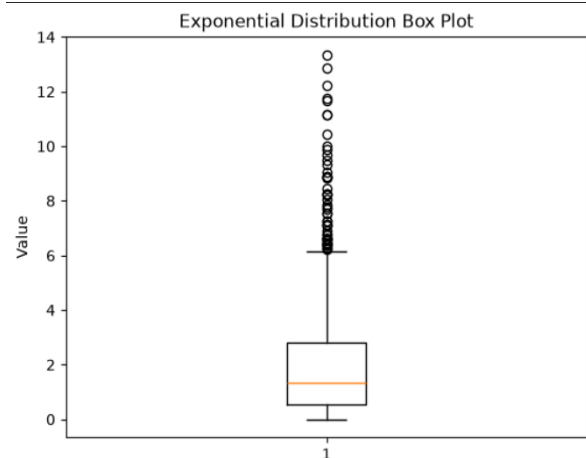
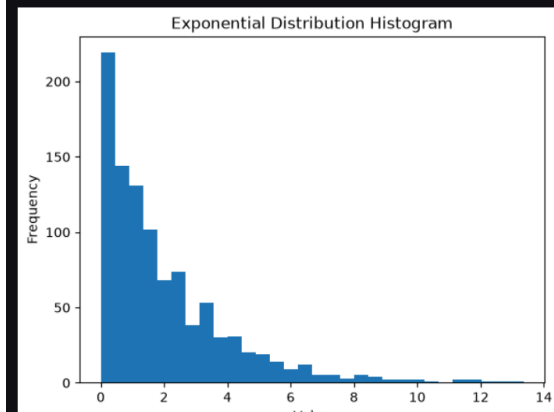


6. Percentage between 40 and 60 = 67.4 %  
7. Percentage between 30 and 70 = 95.0 %

----- UNIFORM DISTRIBUTION -----  
Number of values = 1000  
Minimum = 0.06327226268370012  
Maximum = 99.68719553063302  
Mean = 49.35832918839314



----- EXPONENTIAL DISTRIBUTION -----  
Scale = 2  
Number of values = 1000  
Mean = 2.0242628933112905



```
13. Extreme Values
Q1 = 0.5521766160792787
Q3 = 2.7970092893927796
IQR = 2.2448326733135007
Possible extreme values = 49

14. Percentage greater than 2 = 37.4 %
15. Percentage greater than 5 = 8.6 %
```

### C. Coin Toss Probability Experiment (with prints):

1. Simulate 1,000 coin tosses → calculate experimental probability of Heads & Tails.
2. Compare with theoretical probability ( $\frac{1}{2}$  each).
3. Bar chart: frequency of Heads vs Tails.
4. Pie chart: percentage of Heads vs Tails.
5. If 10 tosses → Heads = 8 → calculate experimental probability of Heads.
6. Simulate 10,000 tosses → complete analysis:
  - Total tosses
  - Number of Heads & Tails
  - Experimental probability (Heads/Tails)
  - Theoretical probability (Heads/Tails)
  - Difference between experimental & theoretical
  - Bar chart
  - Pie chart

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

print("===== C) COIN TOSS EXPERIMENT =====")

# =====
# 1. 1000 COIN TOSSES
# =====

coin = np.random.choice(
    ["Heads", "Tails"],
    1000
)

heads = np.sum(coin == "Heads")
tails = np.sum(coin == "Tails")

heads_probability = heads / 1000
tails_probability = tails / 1000

print("\n1. 1000 Coin Tosses")
```

```

print("Total Tosses =", 1000)
print("Heads =", heads)
print("Tails =", tails)

print("Experimental Probability of Heads =",
      heads_probability)

print("Experimental Probability of Tails =",
      tails_probability)

# =====
# 2. THEORETICAL PROBABILITY
# =====

print("\n2. Theoretical Probability")

print("Theoretical Probability of Heads =",
      0.5)

print("Theoretical Probability of Tails =",
      0.5)

# =====
# 3. BAR CHART
# =====

plt.bar(
    ["Heads", "Tails"],
    [heads, tails]
)

plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.title("Frequency of Heads and Tails")
plt.show()

# =====
# 4. PIE CHART
# =====

plt.pie(
    [heads, tails],
    labels=["Heads", "Tails"],
    autopct="%1.1f%%"
)

```

```

)

plt.title("Percentage of Heads and Tails")
plt.show()

# =====
# 5. 10 TOSSES WITH 8 HEADS
# =====

print("\n5. 10 Tosses with 8 Heads")

heads_10 = 8
total_10 = 10

experimental_probability_10 = heads_10 / total_10

print("Total Tosses =", total_10)
print("Number of Heads =", heads_10)
print("Experimental Probability of Heads =",
      experimental_probability_10)

# =====
# 6. 10,000 COIN TOSSES
# =====

print("\n===== 10,000 COIN TOSSES =====")

coin_10000 = np.random.choice(
    ["Heads", "Tails"],
    10000
)

heads_10000 = np.sum(coin_10000 == "Heads")
tails_10000 = np.sum(coin_10000 == "Tails")

experimental_heads = heads_10000 / 10000
experimental_tails = tails_10000 / 10000

theoretical_heads = 0.5
theoretical_tails = 0.5

difference_heads = abs(
    experimental_heads - theoretical_heads
)

```

```

difference_tails = abs(
    experimental_tails - theoretical_tails
)

# Complete analysis
print("\nComplete Analysis")

print("Total Tosses =", 10000)

print("Number of Heads =", heads_10000)

print("Number of Tails =", tails_10000)

print(
    "Experimental Probability of Heads =",
    experimental_heads
)

print(
    "Experimental Probability of Tails =",
    experimental_tails
)

print(
    "Theoretical Probability of Heads =",
    theoretical_heads
)

print(
    "Theoretical Probability of Tails =",
    theoretical_tails
)

print(
    "Difference in Heads Probability =",
    difference_heads
)

print(
    "Difference in Tails Probability =",
    difference_tails
)

# =====
# BAR CHART - 10,000 TOSSES

```



```
# =====

plt.bar(
    ["Heads", "Tails"],
    [heads_10000, tails_10000]
)

plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.title("10,000 Coin Tosses")
plt.show()

# =====
# PIE CHART - 10,000 TOSSES
# =====

plt.pie(
    [heads_10000, tails_10000],
    labels=["Heads", "Tails"],
    autopct="%1.1f%%"
)

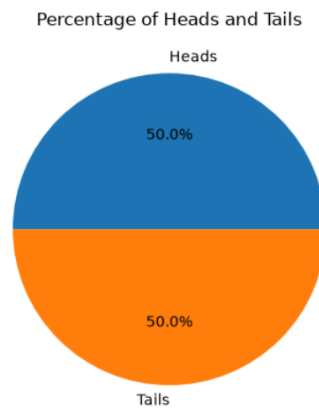
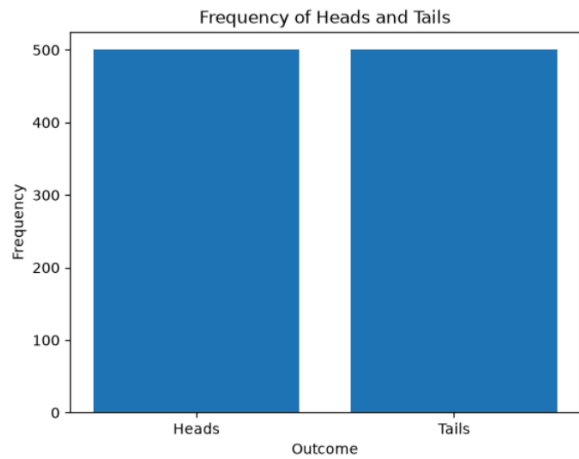
plt.title("10,000 Coin Tosses Percentage")
plt.show()
```

## OUTPUT

```
===== C) COIN TOSS EXPERIMENT =====

1. 1000 Coin Tosses
Total Tosses = 1000
Heads = 500
Tails = 500
Experimental Probability of Heads = 0.5
Experimental Probability of Tails = 0.5

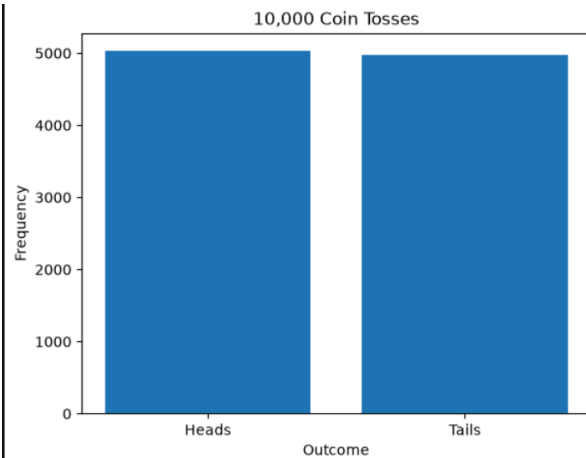
2. Theoretical Probability
Theoretical Probability of Heads = 0.5
Theoretical Probability of Tails = 0.5
```



```
5. 10 Tosses with 8 Heads
Total Tosses = 10
Number of Heads = 8
Experimental Probability of Heads = 0.8

===== 10,000 COIN TOSSES =====

Complete Analysis
Total Tosses = 10000
Number of Heads = 5025
Number of Tails = 4975
Experimental Probability of Heads = 0.5025
Experimental Probability of Tails = 0.4975
Theoretical Probability of Heads = 0.5
Theoretical Probability of Tails = 0.5
Difference in Heads Probability = 0.0024999999999999467
Difference in Tails Probability = 0.0025000000000000022
```



10,000 Coin Tosses Percentage

